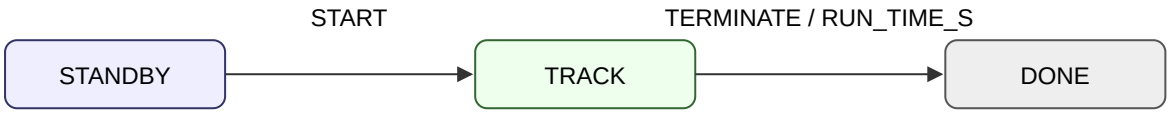
propPitch: how it works

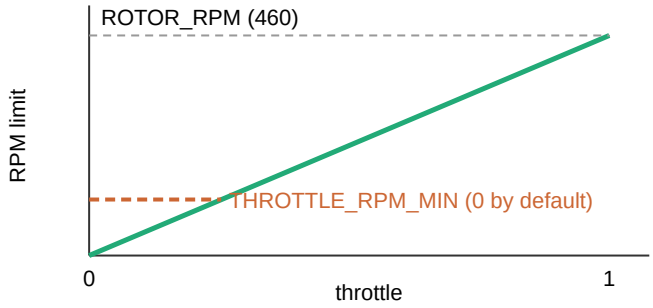
Holds a propeller plane's collective blade pitch at the angle that gives the most forward thrust, computed every tick from the game's own blade aerodynamics, and runs the rotors at the RPM limit the throttle sets. With Brakes on, the props brake the plane instead. It flies only the propellers; the pilot flies the plane. START works the same on the runway or in flight.

Phases



START finds every robotic rotor and the control-surface blades below it (any part with a `Deploy Angle` field), disables the craft's KAL controllers (`DISABLE_KAL`), runs every rotor at the throttle's RPM limit (below) and `ROTOR_TORQUE_PCT` with its motor engaged and unlocked, releases each rotor's brake, and reads each blade's position and axes once (vessel frame, relative to its rotor). Blade part names are looked up in `bladeTable.py`; an unknown blade stops START with its name. TRACK reads the throttle, sets the rotors' RPM limit, and commands the optimum every tick; every `REASSERT_S` it re-asserts the KALs' and rotors' settings and the brakes. TERMINATE re-enables the KALs and restores each rotor's RPM and torque limits and motor state.

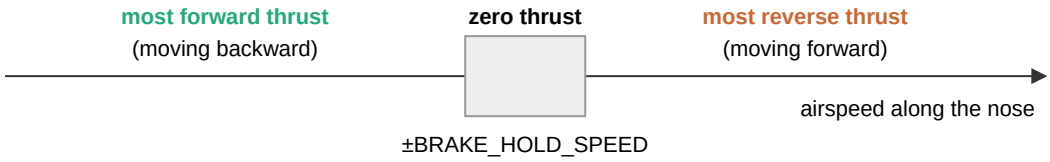
Throttle sets the RPM limit



With `THROTTLE_RPM` on (the default), RPM limit = max(`THROTTLE_RPM_MIN`, throttle × `ROTOR_RPM`).

- The throttle is read in each tick's batch (`vessel.control.throttle`, the main throttle, whatever drives it: keyboard, joystick axis, or a script). START sets the rotors from the throttle at that moment.
- The rotors' `target_rpm` is written only when the throttle's RPM has moved by `RPM_WRITE_STEP` (2 rpm) since the last write, or has reached either end of the range. The re-assert holds the rotors to the last value written.
- The pitch search scores every angle at that RPM limit, so the blades always sit at the best angle for the RPM the throttle asks for. At 0 rpm the blades are scored as a stopped disc.
- A throttle cut stops the rotors on purpose, so the re-assert's brake release (below) is skipped while the limit is at or below `BRAKE_RPM`.
- `THROTTLE_RPM=False` runs every rotor at `ROTOR_RPM` whatever the throttle.

Prop brake (the Brakes group)



What the blades are pitched for while `vessel.control.brakes` is on (`PROP_BRAKE`, default on).

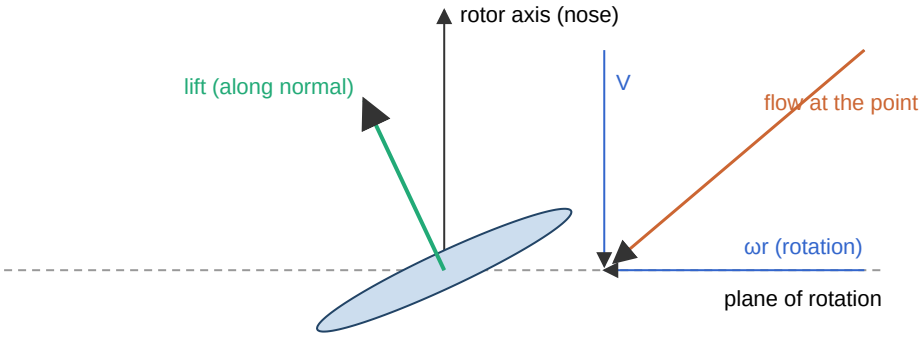
- Every rotor runs at `ROTOR_RPM` whatever the throttle. On the tick Brakes comes on, every rotor's own brake latch (which the craft's Brakes group sets) is released, and again at every re-assert while braking. The log says `prop brake on` / `prop brake off`.
- The sign of the air velocity's nose component (y in the vessel's axes) picks the objective: above `BRAKE_HOLD_SPEED` (0.5 m/s) forward, the angle with the most thrust against the nose; that far backward, the most thrust along it; between, the angle with zero thrust.
- The braking search runs over the blades' whole Deploy Angle range, ±1.5 × ctrlSurfaceRange of the tightest blade (±90° for fan blades and propellers, ±22.5° for heli blades; `bladeTable.py`'s `range`). Negative angles pitch the blade past flat the other way and give reverse thrust. Forward flight's search stays in `PITCH_MIN_DEG`…`PITCH_MAX_DEG`, clipped to the same limits.
- There is no limit on the braking thrust: on `qs_prop_noon` it decelerates at about 4 g, and the pilot lets go of Brakes if the plane starts to go over. Brakes off returns to the throttle's RPM and the forward optimum on the next tick.

Rotor brakes

A rotor's brake is a latched mode inside the game's rotor module. The Brakes action group sets and clears it, and the rotor also latches it when it runs out of electric charge. A braked rotor holds 0 rpm whatever its motor is told, and no kRPC property reads or clears the latch. Only the rotor's own `BrakeAction` does: the pilot sends it Deactivate, which releases that rotor and leaves the vessel's wheel brakes alone.

- At START every rotor's brake is released, whatever the Brakes group is set to. The log says `rotor brakes released: n of m (Brakes group on/off)`.
- At each re-assert, a rotor reading below `BRAKE_RPM` while the Brakes group is off and the RPM limit is above `BRAKE_RPM` has its brake released again. That clears a power-loss latch. With `PROP_BRAKE=False` and the group on, a stopped rotor is left alone, so Brakes stops the rotors.

The blade law (KSP's, evaluated off-line)



One blade, seen along its span. Thrust is the component of lift and drag along the nose. Deploy Angle turns the normal about the span: 0 = normal tangential (a flat plate into the rotation), 90 = flat in the disc.

KSP treats each blade as one lifting surface evaluated at one point: the part's `velocityOffset` along its span (−5.8 m on the R-25, about 6.0 m from the hub). With *u* the direction of that point's velocity through the air, *n* the blade's normal turned by *s*·Deploy Angle about its span, $c = u \cdot n$, $q = \tfrac{1}{2}\rho W^2$ and $M = W/a$ at that point:

$$\text{lift} = -n \cdot \text{sign}(c) \cdot Cl(|c|) \cdot Ml(M) \cdot k \cdot q \cdot 0.036 \quad \text{(at the part's CoL offset)}$$

$$\text{drag} = -u \cdot Cd(|c|) \cdot Md(M) \cdot k \cdot q \cdot 0.015 \quad \text{(at the part's origin)}$$

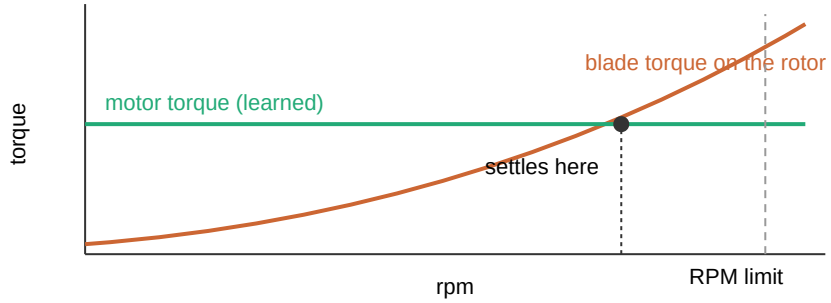
k = deflectionLiftCoeff × ctrlSurfaceArea. The four curves are the part cfg's (FloatCurves, cubic Hermite), and 0.036 and 0.015 are Physics.cfg's lift and lift-drag multipliers. Lift acts along the normal, not perpendicular to the flow. The point's velocity is the vessel's air velocity (read in the vessel's axes through a hybrid frame) plus ω×r, with ω from the RPM limit. Each rotor's blades are evenly spaced, so their geometry at any one moment stands for the average around the disc.

Deflection sense *s*: kRPC does not expose a blade's CW/CCW variant. At START, each blade gets the sign that gives forward thrust at rest (60/75/85°) at `ROTOR_RPM`. Thrust depends only on the product of *s* and the spin direction.

The search

Every tick the pilot totals the thrust along the nose at Deploy Angles `PITCH_MIN_DEG`…`PITCH_MAX_DEG` (0…90) on a 1° grid (`SEARCH_STEP_DEG`), then runs a golden-section search around the best cell to `SEARCH_TOL_DEG`. The angle found is commanded on every blade (only when it moved by 0.01°), and KSP applies a Deploy Angle without actuator delay.

Load and rpm



Each angle is scored at the rpm its rotor would settle at: the RPM limit if the motor holds it, else where the torques balance.

KSP's rotor is a velocity drive (damper 1000) whose torque is capped at maxTorque × torque limit (70 kN·m on the EM-32S at 100%), and is zero without electric charge or with the brake on. Its charge draw, 0.06 × 70 × limit × rpm/460 / 0.75 EC/s, does not depend on load. For each angle the model computes the blades' torque at the RPM limit. If that is more than the rotor's motor torque, the model bisects for the rpm where the two balance and scores the thrust there. If the air drives the rotor (the blades' torque is negative) harder than the motor can hold it back, the rotor overspeeds: the model bisects above the limit for the rpm where the driving torque equals the motor torque.

The motor torque starts at the game's cap and is learned from the measured rpm (`Rotor.observe`). When a rotor runs more than 2% below its limit and its rpm readout has stayed flat for 0.6 s of game time, the model's blade torque at that rpm becomes the motor torque, smoothed over 0.5 s. The game refreshes the readout every 0.3 s, so a rotor spinning up (after START, or after the throttle is opened) reads flat between refreshes, and the 0.6 s window keeps that from counting. Each reading is stamped at the end of its tick, so a tick longer than the window starts a fresh one. While the rotor holds its limit, or runs above it (throttle pulled back), the estimate stays at least the current load and recovers toward the cap over 5 s. A motor without power learns ~0, and the search then picks the angle that loads that rotor least.

The log

field	what
`pitch=` / `opt=`	the angle flown / the model's optimum
`J=`	measured forward aero force per unit mass (thrust less airframe drag), m/s²
`model=`	the model's blade thrust per unit mass at the angle flown
`rpm=a/b`	measured mean rpm / the rpm the model expects the slowest rotor to settle at
`torque=`	the worst rotor's blade torque over the game's cap
`motor=`	learned motor torque per rotor, kN·m
`va=`	velocity through the air in the vessel's axes (x right, y nose, z down)
`thr=`	the throttle read this tick (0…1)
`brake=`	`-` not braking; `rev` / `fwd` / `hold`: what the props are pitched for while braking
`rotor rpm`	an event at START: "from the throttle" or "fixed", and the limit set
`re-set:`	an event: what a re-assert found changed (KAL on, rotor limits, motor off, locked, brake on a stopped rotor)

J is read before the tick's command, so it answers the previous tick's angle. The panel shows the measured rpm and the RPM limit. `tools/bladeprobe.py` replaces the command with a sweep, and `tools/probesum.py` compares J with the model at each angle held. `tools/propfly.py --throttle T` holds the throttle at T (default 1); `--brake-at V` presses Brakes once past V m/s.